



PHP

[Avant de commencer essayez d'installer un server web qui tourne PHP - regardez le guide d'installation. Nous allons voir ensemble si vous arrivez à des problèmes.]

Le langage PHP est un langage de programmation côté serveur. En effet, le terme PHP est l'acronyme de « PHP Hypertext Preprocessor » - oui oui, le premier « P » de PHP est effectivement lui-même l'abréviation de « PHP » (ils sont fous ses programmeurs).

Le PHP est plutôt un langage de scripts orienté objet pour le développement d'applications Web. Il est interprété côté serveur par un interpréteur PHP et utilisé principalement pour générer de façon dynamique de pages HTML. Les pages sont d'abord générées coté serveur (sous forme HTML/JS) et après envoyées aux clients (navigateurs). Donc du côté client, la page reçue est en HTML/JS, il n'a pas de PHP car le navigateur est incapable de l'interpréter.

Le code de PHP s'écrit entre les balise `<?PHP` et `?>` dans un fichier qui a l'extension **.php**. Voici un exemple avec le code PHP (à remarquer que que parti du code est en HTML).

```
html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Test de page avec PHP< </title>
</head>
<body>
  <h2> Page de test de code PHP </h2>
  <?php
    // Ceci est du code PHP
    $date = date("d/m/Y"); # autre commentaire, Variable PHP + fonction
    $heure = date("H:i:s");
    /* Affichage */
    echo "Nous sommes le <strong >$date </strong > et
        il est <strong >$heure </strong >";
  ?>
</body>
</html>
```

Si on tourne cette page dans notre navigateur la page aura des valeurs précis pour heure et date (si vous avez un serveur locale, votre address doit être qqchse comme `http://localhost:8080/test.php`). Vous pouvez aussi inspecter votre page pour voir que dans le code HTML les variables `$date` et `$heure` sont remplacés par des valeurs spécifiques.

PHP est assez proche du langage C avec quelques particularités quand même.

Variables et fonctions en PHP

Pas besoin de déclarer les variables. On les utilise et c'est tout. Le type d'une variable dépend de son utilisation. Il peut même changer au cours du temps - c.à.d. c'est la valeur affectée qui détermine le type.

Pour s'en sortir d'une telle facilité (et découvrir les variables au fur et à mesure qu'elles apparaissent dans le code) PHP a besoin d'un truc. Le nom de toutes les variables commence par le signe \$. Donc une variable commence toujours par \$ suivi du nom de la variable (sensible à la casse). Un nom de variable valide doit commencer par une lettre ou un underscore (_), suivi de lettres, chiffres ou underscores.

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8"> <title>Test de page avec PHP</title>
</head>
<body>
<h2>Page de test de code PHP </h2>

<?php
$a = "Polytech";
$_b = "Paris-Saclay";
$c = 2021;
$c = 2022;
echo "$a $_b $c-$c";
?>

</body>
</html>
```

Après il faut faire un peu attention car avec PHP nous pouvons faire l'affectation par valeur ou par référence (en anglais : pass by value vs. pass by reference).

Affectation par valeur avec \$	Affectation par référence avec &
<pre>\$mavar1="Polytech"; \$mavar2="Paris-Saclay"; \$mavar2=\$mavar1; \$mavar1="APP3"; echo "\$mavar1 \$mavar2";</pre>	<pre>\$mavar1="Polytech"; \$mavar2="Paris-Saclay"; \$mavar2=&\$mavar1; \$mavar1="APP3"; echo "\$mavar1 \$mavar2";</pre>
A la fin du code la variable \$mavar2 contient la chaîne "Polytech", on lui a affecté la valeur de l'expression \$mavar1 quand \$mavar1 était "Polytech". Ensuite on a changé la valeur de \$mavar1.	Ici \$mavar2 devient un alias de la variable \$mavar1. Les modifications sur \$mavar1, sont répercutées sur \$mavar2. Les modifications sur \$mavar2 sont également répercutées sur \$mavar1.

Les fonctions ne peuvent ni ne doivent pas déclarer de type de retour (ni void ni int, ni rien). Pour distinguer le début d'une fonction dans le code PHP a besoin qu'elle commencent par le mot **function**.

Voici un exemple. Ici il n'y a pas de HTML et la fonction `transformeJSON` n'est jamais appelée donc il ne se passe rien. Par ailleurs PHP peut bien sur transformer un fichier JSON (si vous voulez apprendre un peu sur le format json <https://www.json.org/json-en.html>).

```
<?php
function transformeJSON ($nomJSON){
    $jsonString = file_get_contents($nomJSON.".json");
    $data = json_decode($jsonString, true);
    $data["activity_name"] = "TENNIS";
    $newJsonString = json_encode($data);
    file_put_contents($nomJSON."2.json", $newJsonString);
}
?>
```

Mais son plus grand intérêt de PHP est de produire quelque chose sous forme de texte avec les fonction `print()` ou `echo` pour produire du HTML (ou du JSON).

Il y a plusieurs fonctions utiles pour manipuler les chaînes de caractères (une liste à la fin).

Produire du HTML

Dans la langue PHP il existe plusieurs opérateurs d'affectation en plus de `=`, qui fonctionnent comme leur équivalent dans d'autres langages comme le C (`+=`, `-=`, `*=`). Il y a des types scalaires (boolean, integer, etc.) mais aussi des tableaux (array) et objets (object), et bien sûr il y a des structures conditionnelles (if, elseif, else) et des boucles (for, while, et do...while). Nous allons les utiliser pour créer des pages HTML.

Ceci est un exemple de HTML qui affiche une table de multiplication (en le calculant, donc on évite la création à la main) :

```
<!doctype html>
<html lang="fr">
<head>
    <meta charset="utf-8">
    <title>Créer un tableau</title>
</head>
<body>
    <h1>Exemple tableau</h1>
    <table border="1">

<?php
for ($j=1; $j<10; $j++){
    print("<tr>");
    for ($i=1; $i<10; $i++){
        if ($i==1 or $j==1)
            print("<th>".($i*$j)."</th>");
        else
            print("<td>".($i*$j)."</td>");
    }
    print("</tr>");
}
?>
    </table>
</body>
</html>
```

Dans cet exemple on voit bien la partie PHP (entre `<?php` et `?>`) au milieu du HTML. Lorsque le navigateur demande le fichier PHP, PHP est exécuté sur le serveur et c'est le HTML suivant qui est envoyé au client navigateur (visible avec Inspect) :

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>TitreCréer un tableau</title>
</head>
<body>
  <h1>Exemple tableau</h1>
  <table border="1">

<tr><th>1</th><th>2</th><th>3</th><th>4</th><th>5</th><th>6</th><th>7</th>
<th>8</th><th>9</th></tr><tr><th>2</th><td>4</td><td>6</td><td>8</td>
<td>10</td><td>12</td><td>14</td><td>16</td><td>18</td></tr><tr><th>3</th>
<td>6</td><td>9</td><td>12</td><td>15</td><td>18</td><td>21</td><td>24</td>
<td>27</td></tr><tr><th>4</th><td>8</td><td>12</td><td>16</td><td>20</td>
<td>24</td><td>28</td><td>32</td><td>36</td></tr><tr><th>5</th><td>10</td>
<td>15</td><td>20</td><td>25</td><td>30</td><td>35</td><td>40</td><td>45</td>
</tr><tr><th>6</th><td>12</td><td>18</td><td>24</td><td>30</td><td>36</td>
<td>42</td><td>48</td><td>54</td></tr><tr><th>7</th><td>14</td><td>21</td><td>28</td><td>36</td>
<td>42</td><td>49</td><td>56</td><td>63</td></tr><tr><th>8</th><td>16</td><td>24</td><td>32</td><td>40</td><td>48</td>
<td>56</td><td>64</td><td>72</td></tr><tr><th>9</th><td>18</td><td>27</td><td>36</td><td>45</td><td>54</td><td>63</td><td>72</td><td>81</td></tr>
</table>

</body>
</html>
```

Et voici le résultat en HTML, visible avec Inspect dans votre navigateur.

Ce clair que faire ça à la main n'est pas intéressant, donc merci PHP ! Le tableau de l'HTML n'est pas bien indenté mais il s'affiche correctement comme cela dans le navigateur :

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

Si l'intérêt de PHP est de sortir du texte, son autre intérêt est aussi d'avoir des **entrées**. Comme du code PHP est inséré dans une page HTML, ses entrées seront dans l'URL que le navigateur utilise pour charger la page.

Par exemple l'url :

`http://localhost/test-for.php?lignes=13&colonnes=5`

donne 2 variables d'entrée au code PHP : la variable lignes qui vaut 13 et la variable colonnes qui vaut 5 (notez l'ajout `?var1=x&var2=y`).

Mais comment récupérer cela dans PHP? Il n'y a pas de variable \$lignes ou \$colonnes mais par contre un **tableau associatif** \$_GET dans le quel on retrouvera ces variables. Si vous aviez envie d'avoir des variables \$lignes ou \$colonnes, voilà ce qu'il faut rajouter :

Bien sûr rien ne garantie qu'il y a bien les variables lignes et colonnes dans l'url et si elles n'y sont pas le script PHP va planter. On pensera donc à solidifier son code en rajoutant une valeur par défaut :

```
<?php
$lignes = $_GET["lignes"];
$colonnes = $_GET["colonnes"];
// la suite ici ...
```

C'est quoi un tableau associatif comme \$_GET ? C'est un super truc comme un tableau qui associe des valeurs à des clés (qui ne sont pas des numéros qui se suivent comme dans un tableau classique, mais des chaînes de caractères). Comme toute variable en PHP, pas besoin de le déclarer mais il faut savoir :

```
<?php
if (array_key_exists('lignes', $_GET))
    $lignes = $_GET["lignes"];
else
    $lignes = 10;
// la suite ici ...
```

- que le tableau peut contenir toutes sortes de valeurs (nombre, chaîne, booléen, etc.) ;
- que le tableau peut avoir des indices en chiffres mais ces derniers sont convertis en nombre ;
- on peut afficher un tableau "tout d'un coup" avec print_r

```
$toto["valeur"] = 20;
$toto["trouve"] = false;
$toto["remplace"] = 0;
print_r($toto);
```

Voici un tableau avec différentes valeurs et indices (converties en nombre). Et voici comment notre exemple peut mélanger différents tableaux avec des valeurs et indices différents:

```
for ($j=1; $j<=$lignes; $j++){
    print("<tr>");
    for ($i=1; $i<=$colonnes; $i++){
        if ($i==1 or $j==1)
            print("<th>".($i*$j)."</th>");
        else{
            if ( ($i*$j) == $toto["valeur"]){
                $toto["trouve"] = true;
                print("<td><strong>".$toto["remplace"]."</strong></td>");
            }else{
                print("<td>".($i*$j)."</td>");
            }
        }
    }
    print("</tr>");
}
```

Une alternative des variables \$_GET sont les variables \$_POST, mais ici l'information qu'on envoie n'est pas visible dans notre url (donc un peu plus sécurisé !)

À partir d'ici une référence de PHP.

PHP Opérateurs d'affectation, types

Il existe plusieurs opérateurs d'affectation en plus de =, qui fonctionnent comme leur équivalent dans d'autres langages comme le C :

+=, -=, *=, /=, %= (modulo puis affectation), .= (concaténation puis affectation)

Types de données :

Types scalaires de base :

Entiers (integer) ;

Flottants (double ou float) ;

Chaînes de caractères (string) ;

Booléens (boolean) qui contiennent les valeurs TRUE ou FALSE

Types composés :

tableaux (array)

objets (object)

Plus d'info ici : <https://www.php.net/manual/fr/language.variables.basics.php>

Instructions conditionnelles & Boucles

Les structures conditionnelles communes, if, if else sont disponibles mais également elseif, par exemple :

```
if (conditionA) {  
    // instructions A  
} elseif (conditionB1 && conditionB2) {  
    // instructions B  
} elseif (conditionC1 || conditionC2) {  
    // instructions C  
} else {  
    // instructions D  
}
```

Les boucles for, while et do-while :

```
for ($i=0 ; $i < $n ; $i++) {  
    // instructions  
}  
  
while (condition) {  
    // instructions  
}  
  
do {  
    // instructions  
} while (condition);
```

Tableaux

```
$tab = array("A", "B", "C");
$tab[0] = "A";
$tab[1] = "B";
$tab[2] = "C";
echo $tab[1]; // affiche "B"
```

Parcours de tableaux, 2 méthodes :

```
for ($i = 0 ; $i < count($tab) ; $i++) {
    echo "La valeur $i est $tab[$i] <br>";
}
echo "<ul>";
```

ou

```
foreach ($tab as $valeur) {
    echo "<li> $valeur </li>";
}
echo "</ul>";
```

Remarque : valeurs pas toutes de même type nécessairement !!!!

Tableaux associatifs ou dictionnaires

Les clés peuvent être des entiers ou des chaînes :

```
$notes = array("Marie" => 14, "Paul" => 12, "Pierre" => 7);
$notes["Marie"] = 14;
$notes["Paul"] = 12;
$notes["Pierre"] = 7;
```

2 méthodes de parcours :

```
echo "<table >";
foreach ($notes as $clef => $valeur) {
    echo "<tr><td>$clef</td><td>$valeur</td></tr>\n";
}
echo "</table>";
```

ou

```
echo "<table >";
reset($notes);
while (next($notes)) { // il y a aussi prev
    $etud = key($notes);
    $note = current($notes);
    echo "<tr><td>$etud</td><td>$note</td></tr>\n";
}
echo "</table>";
```

Fonctions

```
function bonjour($nom) {  
    echo "Bonjour $nom !<br>";  
}  
bonjour("Marie");
```

```
function somme($x, $y) {  
    return $x + $y;  
}  
$z = somme(3, 10);
```

```
function coordonnees($ville) {  
    ...  
    return array($x, $y);  
}  
list($a, $b) = coordonnees("Tokyo");
```

Fonctions prédéfinies utiles

<code>empty(\$v) :</code>	teste si \$v est définie est a une valeur non nulle
<code>isset(\$v) :</code>	teste si \$v est définie
<code>abs(\$v) :</code>	valeur absolue
<code>ceil(\$v) :</code>	entier immédiatement supérieur
<code>floor(\$v) :</code>	entier immédiatement inférieur
<code>round(\$f) :</code>	arrondi
<code>max(\$v1, \$v2, ..., \$vn) :</code>	plus grande valeur
<code>min(\$v1, \$v2, ..., \$vn) :</code>	plus petite valeur
<code>rand() :</code>	valeur aléatoire
<code>date(\$format) :</code>	renvoie la date courante formatée. Exemples : "d/m/Y", "H:i:s"

Fonctions prédéfinies utiles pour les **chaînes** des characters

`explode($sep, $chaine)` : divise `$chaine` selon le séparateur `$sep` et renvoie les valeurs dans un tableau

`implode($sep, $tableau)` : regroupe les valeurs de `$tableau` avec le séparateur `$sep` et renvoie la chaîne de caractères

`htmlspecialchars($ch)` : retourne la chaîne `$ch` dans laquelle les caractères réservés au HTML ('<', '>', '&'...) ont été remplacés par leur code HTML.

`strlen($chaine)` : renvoie la taille de la chaîne de caractères

`strstr($ch1, $ch2)` : envoie le contenu de `$ch1` à partir de la première occurrence de `$ch2` (sinon, renvoie faux)

`substr($ch, $i, $l)` : renvoie la sous-chaîne de `$ch` de longueur `$l` à partir de l'index `$i`

Voici quelques exemples d'utilisation :

```
$fruits = "abricot | kiwi | pomme | fraise | banane";
```

```
$liste_fruits = explode('|', $fruits); // => créer un tableau avec 6 strings (les fruits)
```

```
$fruits2 = implode(' ', $liste_fruits); // => recombine les 6 strings avec espace entre eux
```

```
echo $fruits2 // => "abricot kiwi pomme fraise banane"
```

```
$fruits = "abricot | kiwi | pomme | fraise | banane";
```

```
$tmp = strstr($fruits, '|'); echo $tmp; // => "| kiwi | pomme | fraise | banane"
```

```
$tmp2 = substr($fruits, 8, 4); echo $tmp2; // => "kiwi"
```

Fonctions prédéfinies utiles (**tableaux**)

`is_array($tab)` : teste si `$tab` est un tableau

`count($tab)` : nombre d'éléments du tableau

`sort($tab)` : tri du tableau sur les valeurs, ordre ascendant

`rsort($tab)` : tri du tableau sur les valeurs, ordre descendant

`ksort($tab)` : tri du tableau sur les clés, ordre ascendant

`krsort($tab)` : tri du tableau sur les clés, ordre descendant

`max($tab)` : plus grande valeur

`min($tab)` : plus petite valeur

Fonctions prédéfinies utiles (tableaux)

`$_SERVER` : informations sur le serveur et l'exécution en cours (heure, hôte, adresse IP, referer, nom du script, etc.)

`$_GET` : variables GET

`$_POST` : variables POST

`$_COOKIES` : cookies

`$_REQUEST` : agrégation de `$_GET`, `$_POST` et `$_COOKIES` `$_FILES` : variables des fichiers téléchargés

`$_SESSION` : variables de session `$_ENV` : variables d'environnement

Pour plus de détails, Cf.

<http://www.php.net/manual/fr/reserved.variables.php>

Liens utiles

1. PHP tuto par l'équipe de documentation PHP <https://www.php.net/manual/en/index.php>
2. Tuto interactive du w3schools <https://www.w3schools.com/php/>
3. En Français. Cours de P.Giraud sur PHP et SQL : <https://www.pierre-giraud.com/php-mysql-apprendre-coder-cours/>
4. Internet ...